EXPERIMENT – 1

Aim: To understand the principles and practices of DevOps and the role and responsibilities of a DevOps Engineer.

Objectives

1. To understand the basic concepts, principles, and importance of DevOps.
2. To study various DevOps practices such as Continuous Integration, Continuous Delivery, automation, monitoring, and collaboration.
3. To understand the role, responsibilities, and skills required for a DevOps Engineer.

Description

1. DevOps

DevOps is a combination of Development (Dev) and Operations (Ops). It is a software development approach that focuses on collaboration, automation, continuous integration, continuous delivery, and continuous monitoring.

DevOps helps organizations develop, test, and deliver software faster, more reliably, and efficiently.

2. DevOps Lifecycle

The DevOps lifecycle consists of different stages that form a continuous cycle. The main stages are:

1. Plan: Planning project requirements, features, and tasks.
2. Code: Writing and developing application source code.
3. Build: Converting source code into an executable application.
4. Test: Testing the application to identify and fix errors.
5. Release: Preparing the application for deployment.
6. Deploy: Deploying the application to the production environment.
7. Operate: Managing and maintaining the application.
8. Monitor: Monitoring application performance and identifying issues.

3. Principles of DevOps

The main principles of DevOps are:

- Collaboration: Development and operations teams work together.
- Automation: Repetitive tasks are automated to reduce manual effort.
- Continuous Integration (CI): Code changes are frequently integrated and tested.
- Continuous Delivery (CD): Software is continuously prepared for deployment.
- Continuous Monitoring: Application performance and system health are monitored.
- Continuous Improvement: Feedback is used to improve the development process.

4. DevOps Practices

The important DevOps practices include:

1. Version Control: Managing and tracking changes in source code.
2. Continuous Integration: Frequently integrating code changes and running automated tests.
3. Continuous Delivery/Deployment: Automating the software release and deployment process.
4. Infrastructure as Code (IaC): Managing infrastructure using configuration files.
5. Automated Testing: Testing applications automatically to identify errors.
6. Containerization: Packaging applications and dependencies into containers.
7. Monitoring and Logging: Tracking application performance and system activities.

5. Role and Responsibilities of a DevOps Engineer

A DevOps Engineer connects development and operations teams and ensures smooth software delivery.

The main responsibilities are:

- Develop and maintain CI/CD pipelines.
- Automate software building, testing, and deployment.
- Manage servers and cloud infrastructure.
- Configure and maintain development environments.
- Monitor application performance.
- Troubleshoot system and deployment issues.
- Ensure application security, reliability, and availability.
- Collaborate with developers and operations teams.

6. Skills Required for a DevOps Engineer

- Linux and operating system knowledge.
- Basic programming and scripting.
- Version control using Git.
- CI/CD pipeline management.
- Cloud computing knowledge.
- Containerization and orchestration.
- Monitoring and troubleshooting.
- Communication and collaboration skills.

7. Tools Used in DevOps

Tool	Purpose
Git/GitHub	Version control
Jenkins	CI/CD automation
Docker	Containerization
Kubernetes	Container orchestration
Ansible	Configuration management
Terraform	Infrastructure as Code
Prometheus	Monitoring
Grafana	Data visualization

Procedure

1. Study the basic concepts and importance of DevOps.
2. Understand the different stages of the DevOps lifecycle.
3. Study the major principles of DevOps, including collaboration, automation, and continuous improvement.
4. Understand DevOps practices such as CI/CD, version control, automated testing, and monitoring.
5. Identify the roles and responsibilities of a DevOps Engineer.
6. Study commonly used DevOps tools and their applications.

Result

The principles and practices of DevOps were studied successfully. The DevOps lifecycle, various DevOps practices, commonly used tools, and the roles and responsibilities of a DevOps Engineer were understood.

Conclusion

DevOps is a combination of culture, collaboration, automation, and continuous improvement. It helps organizations deliver software faster and more reliably. A DevOps Engineer plays an important role in automating software delivery, managing infrastructure, monitoring applications, and ensuring reliable deployments.

EXPERIMENT – 2

Aim: To explore the Version Control System tools for Source Code Management.

Objectives

1. Understand the concepts of VCS: local and remote repositories.
2. Learn basic Git commands for repository management.
3. Understand how to commit, push, pull, and clone repositories.

Procedure – Step-by-step execution in Ubuntu

Step 1: Install Git

First, open the Ubuntu Terminal.

Shortcut: `Ctrl + Alt + T`

Update the package list:

```
None
sudo apt update
```

Install Git:

```
None
sudo apt install git -y
```

Verify the installation:

```
None
git --version
```

Expected output:

```
None
git version 2.x.x
```

(The exact version may differ.)

Step 2: Configure Git

Before using Git, configure your username and email.

```
None
git config --global user.name "Your Name"
```

```
None
git config --global user.email "your-email@gmail.com"
```

Verify the configuration:

```
None
git config --list
```

You should see your configured username and email.

Why? Git uses these details to identify the person who makes each commit.

Step 3: Create a project folder

Create a folder named `git-demo`.

```
None
mkdir git-demo
```

Navigate into the folder:

```
None
cd git-demo
```

Check the current directory:

```
None
pwd
```

Expected output will look like:

```
None
/home/yourusername/git-demo
```

Step 4: Initialize a Git repository

Execute:

```
None
git init
```

Expected output:

```
None
Initialized empty Git repository in /home/yourusername/git-demo/.git/
```

This initializes the current folder as a Git repository.

Remember: `git init` creates a hidden `.git` directory that stores version-control information.

Step 5: Create a sample file

Create a file named `hello.txt`:

```
None
echo "Hello, Welcome to Git Version Control" > hello.txt
```

Display its contents:

```
None
cat hello.txt
```

Expected output:

```
None
Hello, Welcome to Git Version Control
```

Check the repository status:

```
None
git status
```

Expected output will show:

```
None
On branch master

No commits yet

Untracked files:
  hello.txt
```

This means Git has detected the file but is not tracking it yet.

Step 6: Add files to the staging area

Add the file to Git's staging area:

```
None
git add hello.txt
```

Or add all files:

```
None
git add .
```

Check the status:

```
None
git status
```

Expected output:

```
None
Changes to be committed:
  new file: hello.txt
```

Meaning: `git add` moves files into the staging area, preparing them for a commit.

Step 7: Commit changes

Execute:

```
None
git commit -m "Initial commit"
```

Expected output:

```
None
[master (root-commit) abc1234] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt
```

The commit permanently records the current version of the files in the local repository.

Rename the branch to `main`:

```
None
git branch -M main
```

Check the branch:

```
None
git branch
```

Expected output:

```
None
* main
```

Step 8: Create a repository on GitHub

Getting Started with GitHub
eecs130.github.io

1. Open GitHub.
2. Log in to your GitHub account.
3. Click the + icon in the top-right corner.
4. Select New repository.
5. Enter repository name: `git-demo`.
6. Select Public or Private.
7. Click Create repository.

Important: Create an empty repository. Do not select Add README, .gitignore, or License, because we already have a local repository.

Step 9: Connect the local repository to GitHub

After creating the GitHub repository, copy its HTTPS URL.

It will look like:

```
None
https://github.com/your-username/git-demo.git
```

Now return to Ubuntu Terminal.

Connect the local repository to GitHub:

```
None
git remote add origin https://github.com/your-username/git-demo.git
```

Replace `your-username` with your actual GitHub username.

Verify the connection:

```
None
git remote -v
```

Expected output:

```
None
origin  https://github.com/your-username/git-demo.git (fetch)
origin  https://github.com/your-username/git-demo.git (push)
```

Meaning:

- `origin` is the nickname of the remote repository.
- `fetch` is used to retrieve changes.
- `push` is used to upload changes.

Step 10: Push code to GitHub

Execute:

```
None
git push -u origin main
```

Expected output (similar):

```
None
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), done.
To https://github.com/your-username/git-demo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Now open your GitHub repository in the browser and refresh it.

You should see:

```
None
git-demo
    |
    └── hello.txt
```

Push completed

Important about login: GitHub does not accept your normal account password for Git operations over HTTPS. If authentication is requested, use a Personal Access Token (PAT) or configure SSH authentication. Never share your token with anyone.

Step 11: Modify the file and commit again

Now modify the existing file:

```
None
echo "Learning Git commands in Ubuntu" >> hello.txt
```

Check the contents:

```
None
cat hello.txt
```

Expected output:

```
None
Hello, Welcome to Git Version Control
Learning Git commands in Ubuntu
```

Check the status:

```
None
git status
```

Add and commit the changes:

```
None
git add .
```

```
None
git commit -m "Updated hello file"
```

Push the changes:

```
None
git push
```

Now the updated file is available in GitHub.

Step 12: Pull changes from GitHub

To demonstrate pulling, first make a change directly in GitHub:

1. Open your `git-demo` repository.
2. Open `hello.txt`.
3. Click the pencil (Edit) icon.
4. Add another line:

```
None
This change was made on GitHub.
```

5. Click Commit changes.

Now return to Ubuntu Terminal and execute:

```
None
git pull origin main
```

Expected output:

```
None
From https://github.com/your-username/git-demo
 * branch main -> FETCH_HEAD
Updating ...
Fast-forward
 hello.txt | 1 +
```

Verify the updated file:

```
None
cat hello.txt
```

You should now see all three lines, including the change made on GitHub.

Meaning: `git pull` downloads changes from the remote repository and integrates them into the local repository.

Step 13: Clone a repository

Cloning means downloading an existing remote repository to your local computer.

Move outside the current project directory:

```
None
cd ..
```

Clone the GitHub repository:

```
None
git clone https://github.com/your-username/git-demo.git
```

Navigate into the cloned folder:

```
None
cd git-demo
```

Check the files:

```
None
ls
```

Expected output:

```
None
hello.txt
```

This demonstrates how to obtain an existing project from GitHub.

Important: Since your original `git-demo` folder already exists, use a different folder name while cloning:

```
None
cd ..
git clone https://github.com/your-username/git-demo.git git-demo-clone
cd git-demo-clone
ls
```

This avoids the error that occurs when cloning into an existing directory.

Theory

a) What are Version Control Systems?

A Version Control System (VCS) is a tool that tracks changes in source code over time, enabling multiple people to collaborate efficiently.

It helps in:

- Storing different versions of code.
- Rolling back to earlier versions if needed.
- Managing contributions from multiple developers.

b) Types of Version Control Systems

1. Local VCS: Stores versions in a local database (e.g., RCS).
2. Centralized VCS (CVCS): A single server stores all versions, and clients check out files (e.g., SVN).
3. Distributed VCS (DVCS): Every developer has a complete copy of the repository (e.g., Git, Mercurial).

c) Popular VCS Tools for SCM

- Git: Widely used distributed version control system.
- Apache Subversion (SVN): Centralized version control system.
- Mercurial: Similar to Git but simpler command set.
- Perforce: Often used in game development and large binary file storage.

d) Comparison Table

Feature	Git	SVN	Mercurial
Storage	Distributed	Centralized	Distributed
Speed	Very fast	Slower	Fast
Offline work	Yes	Limited	Yes
Popularity	Very high	Medium	Low
Branching	Easy	More complex	Easy

e) Why is Git Preferred in DevOps?

- Distributed architecture enables faster collaboration.
- Lightweight branching and merging make feature development easier.
- Open-source and widely adopted in CI/CD pipelines.
- Offline capability allows work without constant internet.
- Integrates with GitHub, GitLab and Bitbucket.

f) Best Practices in VCS

- Commit frequently with clear messages.
- Use branches for new features or bug fixes.
- Pull the latest changes before starting new work.
- Avoid committing sensitive data.
- Keep repositories organized.
- Use meaningful commit messages.

Conclusion

Version Control Systems like Git are essential in modern software development for tracking changes, enabling collaboration, and ensuring code quality. In this experiment, Git was installed and configured in Ubuntu, a local repository was created, and files were added, committed, pushed, pulled, and cloned using GitHub. Thus, the basic concepts and operations of Version Control Systems for Source Code Management were successfully explored.

EXPERIMENT – 3

Title: Install Git and create a GitHub account and execute various Git operations.

1. AIM: To install Git, create a GitHub account, and execute various Git operations in Ubuntu Linux.

2. OBJECTIVES

- To install Git and configure user credentials.
- To create a GitHub account and repository.
- To perform basic Git operations such as clone, add, commit, push, and pull.
- To understand the interaction between a local repository and a remote GitHub repository.

4. THEORY

Git: Git is a distributed version control system used to track changes in source code. It allows developers to maintain different versions of a project and collaborate with other developers.

GitHub: GitHub is a cloud-based platform used to host Git repositories and collaborate on software development projects.

Git Operations:

Operation	Description
Clone	Downloads a remote repository to the local system.
Add	Adds files to the staging area.
Commit	Saves changes in the local repository.
Push	Uploads local commits to GitHub.
Pull	Downloads and integrates remote changes.
Status	Displays the current repository status.

5. PROCEDURE – COMPLETE EXECUTION

Follow these steps in the same order in your Ubuntu system.

Step 1: Open Ubuntu Terminal

Step 2: Update Ubuntu packages

Execute:

```
None
sudo apt update
```

- Enter your Ubuntu password if requested.
- Press Enter.
- Wait until the update completes.

Note: While entering the password, characters will not appear. This is normal in Linux.

Step 3: Install Git

Execute:

```
None
sudo apt install git -y
```

This installs Git in Ubuntu.

Wait until the installation finishes.

Step 4: Verify Git installation

Execute:

```
None
git --version
```

Expected output will look similar to:

```
None
git version 2.x.x
```

Git successfully installed

Step 5: Configure Git username and email

First, configure your username:

```
None
git config --global user.name "Your Name"
```

Example:

```
None
git config --global user.name "Keerthi"
```

Now configure your email:

```
None
git config --global user.email "your-email@gmail.com"
```

Use the email associated with your GitHub account.

Verify the configuration:

```
None
git config --global --list
```

Expected output:

```
None
user.name=Keerthi
user.email=your-email@gmail.com
```

Remember: Replace the example name and email with your own details.

Step 6: Create a GitHub account

1. Open Firefox or another browser.
2. Visit GitHub.
3. Click Sign up.
4. Enter your email address.
5. Create a password.
6. Select a username.
7. Complete verification.
8. Log in to your GitHub account.

GitHub account created

Step 7: Create a new GitHub repository

1. Log in to GitHub.
2. Click the + icon at the top-right corner.
3. Select New repository.
4. Enter the repository name:

```
None
git-demo
```

5. Select Public.
6. Leave the README, .gitignore and license options unchecked.
7. Click Create repository.

Your repository will be created.

Step 8: Copy the repository URL

On the newly created repository page:

1. Click the green Code button.
2. Select HTTPS.
3. Copy the repository URL.

It will look like:

```
None
https://github.com/USERNAME/git-demo.git
```

Replace `USERNAME` with your actual GitHub username.

Keep this URL ready.

Step 9: Clone the repository into Ubuntu

Go back to Ubuntu Terminal.

Execute:

```
None
git clone https://github.com/USERNAME/git-demo.git
```

Use your actual repository URL.

Example:

```
None
git clone https://github.com/keerthi/git-demo.git
```

Expected output:

```
None
Cloning into 'git-demo'...
warning: You appear to have cloned an empty repository.
```

This warning is normal because we created an empty repository.

What happened?

The GitHub repository has been downloaded to your Ubuntu system.

Step 10: Enter the repository directory

Execute:

```
None
cd git-demo
```

Now you are inside your project directory.

Check the current location:

```
None
pwd
```

Check repository status:

```
None
git status
```

Expected output:

```
None
On branch main

No commits yet

nothing to commit
```

Step 11: Create a file

We will create a file named `README.md`.

Execute:

```
None
touch README.md
```

Now add content to the file:

```
None
echo "Hello World! This is my first Git experiment." > README.md
```

Display the contents:

```
None
cat README.md
```

Expected output:

```
None
Hello World! This is my first Git experiment.
```

Step 12: Check repository status

Execute:

```
None
git status
```

Expected output:

```
None
On branch main

Untracked files:
  README.md

nothing added to commit but untracked files present
```

Meaning: Git detected a new file, but it has not yet been added to the staging area.

Step 13: Stage the changes using Git Add

Execute:

```
None
git add README.md
```

Now verify:

```
None
git status
```

Expected output:

```
None
Changes to be committed:
  new file: README.md
```

Meaning: The file is now in the staging area and ready to be committed.

Step 14: Commit the changes

Execute:

```
None
git commit -m "Add README file"
```

Expected output:

```
None
[main xxxxxxx] Add README file
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

Verify the commit:

```
None
git log --oneline
```

You should see your newly created commit.

Meaning: Your changes are now saved in the local Git repository.

Step 15: Push changes to GitHub

Execute:

```
None
git push origin main
```

Here:

- `push` uploads changes.
- `origin` represents the remote repository.
- `main` is the branch name.

Expected output:

```
None
Enumerating objects: ...
Counting objects: ...
Writing objects: ...
To https://github.com/USERNAME/git-demo.git
   main -> main
```

Changes uploaded to GitHub

Important authentication note:

Step 16: Verify the uploaded file on GitHub

1. Open your GitHub repository in the browser.
2. Refresh the page.
3. Check whether `README.md` is visible.
4. Click the file to view its content.

You should see:

```
None
Hello World! This is my first Git experiment.
```

This confirms that the push operation was successful.

Step 17: Perform Git Pull operation

Now we will make a change directly on GitHub and download it to Ubuntu.

1. Open your GitHub repository.
2. Click `README.md`.
3. Click the pencil (Edit) icon.
4. Add the following new line:

```
None
Changes made directly on GitHub.
```

5. Click Commit changes.

Now return to Ubuntu Terminal.

Execute:

```
None
git pull origin main
```

Expected output:

```
None
Updating ...
Fast-forward
 README.md | 1 +
```

This downloads the changes made on GitHub into your local repository.

Step 18: Verify the pulled changes

Execute:

```
None
cat README.md
```

Expected output:

```
None
Hello World! This is my first Git experiment.
Changes made directly on GitHub.
```

Git pull successfully completed

Finally, execute:

```
None
git status
```

Expected output:

```
None
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

This confirms that the local and remote repositories are synchronized.

7. RESULT

Git was successfully installed and configured in Ubuntu Linux. A GitHub account and repository were created. Various Git operations such as clone, add, commit, push, and pull were performed successfully. The interaction between the local repository and remote GitHub repository was demonstrated.

8. CONCLUSION

Git is a distributed version control system used to track and manage changes in source code. GitHub provides a remote platform for storing and collaborating on Git repositories. By performing various Git operations, we understood how to manage project versions and synchronize changes between local and remote repositories.

Experiment 4: Installing and Configuring Jenkins to Set Up a Build Job

1. Aim: To install and configure Jenkins on Ubuntu Linux and create and execute a build job to understand the concepts of Continuous Integration (CI).

2. Objectives

- To learn the basics of Continuous Integration using Jenkins.
- To install and configure Jenkins on a standalone Ubuntu system.
- To create and execute a simple build job in Jenkins.
- To verify the build output and understand Jenkins job execution flow.
- To gain practical knowledge of job scheduling, console output, and CI pipelines.

4. Theory

Jenkins is an open-source automation server used for Continuous Integration and Continuous Delivery (CI/CD). It helps developers automate tasks such as building, testing, and deploying software applications.

Continuous Integration (CI) is a software development practice in which developers frequently integrate their code changes into a shared repository. Each integration can trigger an automated build and testing process to identify errors early.

Working of Jenkins:

1. Developers make changes to the source code.
2. Code is stored in a version control system such as Git.
3. Jenkins detects or receives the changes.
4. Jenkins executes the configured build job.
5. The build output is displayed in the console.
6. Developers can verify whether the build was successful or failed.

Important Jenkins Components:

- Jenkins Server: Manages build jobs and automation tasks.
- Job: A task configured to perform a particular operation.
- Build: An execution of a Jenkins job.
- Workspace: Directory where Jenkins executes job commands.
- Console Output: Displays logs and results of a build.
- Plugins: Extend Jenkins functionality.

5. Procedure

1. Open the Ubuntu terminal.
2. Update the Ubuntu package repository.
3. Install Java and verify its installation.
4. Add the official Jenkins repository.
5. Install Jenkins using the APT package manager.
6. Start and enable the Jenkins service.
7. Check whether Jenkins is running successfully.
8. Open Jenkins in a web browser using `http://localhost:8080`.
9. Unlock Jenkins using the initial administrator password.
10. Install the suggested plugins.
11. Create an administrator account.
12. Create a new Freestyle Project.
13. Add a build step using the Execute Shell option.
14. Enter a shell command to display a message.
15. Save and execute the job.
16. Verify the build result using Console Output.

6. Execution Steps

Step 1: Update Ubuntu packages

Open the terminal and execute:

```
None
sudo apt update
```

This command updates the package information available in Ubuntu.

Step 2: Install Java

Jenkins requires Java to run. Install OpenJDK 21:

```
None
sudo apt install fontconfig openjdk-21-jre -y
```

Verify Java installation:

```
None
java -version
```

Expected output (version details may differ):

```
None
openjdk version "21..."
OpenJDK Runtime Environment
OpenJDK 64-Bit Server VM
```

Step 3: Add the Jenkins repository

Download the official Jenkins repository signing key:

```
None
sudo mkdir -p /etc/apt/keyrings
```

```
None
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
```

Add the Jenkins LTS repository:

```
None
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | \
sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
```

Update the package repository:

```
None
sudo apt update
```

Step 4: Install Jenkins

Execute:

```
None
sudo apt install jenkins -y
```

This installs Jenkins on the Ubuntu system.

Step 5: Start and enable Jenkins

Start the Jenkins service:

```
None
sudo systemctl start jenkins
```

Enable Jenkins to start automatically whenever the system boots:

```
None
sudo systemctl enable jenkins
```

Check the service status:

```
None
sudo systemctl status jenkins
```

Expected result:

```
None
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded
     Active: active (running)
```

Press `q` to exit the status screen.

Jenkins service running

Step 6: Access Jenkins Dashboard

1. Open Firefox or Chrome in Ubuntu.
2. Enter the following address:

```
http://localhost:8080
```

3. The Jenkins Unlock page will appear.

Step 7: Unlock Jenkins

To obtain the initial administrator password, execute the following command in the terminal:

```
None
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

- Copy the displayed password.
- Paste it into the Administrator Password field in the browser.
- Click Continue.

Step 8: Install Suggested Plugins

- Select Install suggested plugins.
- Wait until the plugins are downloaded and installed.
- Create your first administrator account.
- Enter the username, password, full name and email address.
- Click Save and Continue.
- Click Start using Jenkins.

The Jenkins Dashboard will now appear.

Step 9: Create a New Jenkins Job

1. Click New Item on the Jenkins Dashboard.
2. Enter the project name:

```
MyFirstBuild
```

3. Select Freestyle project.
4. Click OK.

Step 10: Configure the Build Step

1. Scroll down to the Build Steps section.
2. Click Add build step.
3. Select Execute shell.

Important for Ubuntu

In Ubuntu, select Execute shell, not Execute Windows Batch Command. The latter is meant for Windows.

Enter the following command in the Command box:

```
None
echo "Hello Jenkins - My First Build Job"
```

Click Save.

Step 11: Run the Jenkins Job

1. Open the `MyFirstBuild` project.
2. Click Build Now.
3. A new build number will appear under Build History.
4. Click the latest build number.
5. Select Console Output.
6. Observe the execution result.

Step 12: Verify the Build Output

The console output should display:

```
None
Started by user admin
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/MyFirstBuild

[MyFirstBuild] $ /bin/sh -xe /tmp/jenkins....sh

+ echo Hello Jenkins - My First Build Job
Hello Jenkins - My First Build Job

Finished: SUCCESS
```

Build successful

Note: The exact console output may vary depending on the Jenkins version and username.

7. Sample Output

Expected Output:

```
None
Hello Jenkins - My First Build Job
Finished: SUCCESS
```

8. Result

Jenkins was successfully installed and configured on Ubuntu Linux. A Freestyle Project named `MyFirstBuild` was created and executed successfully using the Execute Shell build step. The output was verified through the Jenkins Console Output, demonstrating the basic Continuous Integration workflow.

9. Conclusion

The experiment demonstrated the installation and configuration of Jenkins on Ubuntu Linux and the creation and execution of a simple build job. It provided practical knowledge of Continuous Integration, automated build execution, Jenkins workspace, and console output. Thus, Jenkins helps automate software development tasks and improves the efficiency of the software development process.

Exp-5

Remember these five steps:

1. Jenkins Master manages the jobs.
2. We create an Agent node in Jenkins.
3. We download `agent.jar`.
4. We connect the Agent to Master using a secret key.

5. Master assigns a job to the Agent, and Agent executes it.

Important terms to remember:

Term	Meaning
Master	Manages jobs
Slave/Agent	Executes jobs
agent.jar	Connects agent to Jenkins
Secret key	Authenticates the agent
Label	Identifies the agent
Executor	Number of jobs an agent can run simultaneously

Part 2: Practical execution – Commands

Now let's understand what we actually type in the terminal.

Prerequisite: Jenkins must already be installed and running from Experiment 4.

Step 1: Check Jenkins Master

```
None
sudo systemctl status jenkins
```

Purpose: Checks whether the Jenkins Master is running.

Expected: `active (running)`.

Step 2: Install Java

```
None
sudo apt update
sudo apt install fontconfig openjdk-17-jdk -y
```

Verify:

```
None
java -version
```

Purpose: Java is required to run Jenkins Agent.

Step 3: Create an Agent directory

```
None
mkdir -p ~/jenkins-agent
cd ~/jenkins-agent
```

Purpose: Creates a separate working directory for the Agent.

Step 4: Download Agent JAR

```
None
curl -O http://localhost:8080/jnlpJars/agent.jar
```

Purpose: Downloads the Jenkins Agent program.

Verify:

```
None
ls
```

Expected:

```
None
agent.jar
```

Step 5: Create Agent in Jenkins Dashboard

Open:

```
None
http://localhost:8080
```

Navigate to:

Manage Jenkins → Nodes → New Node

Enter:

Field	Value

Node name	slave1
Type	Permanent Agent
Executors	1
Remote root directory	`/home/yourusername/jenkins-agent`
Labels	slave1
Launch method	Launch agents by connecting them to the controller

Save it.

Step 6: Connect Agent to Master

Open:

Manage Jenkins → Nodes → slave1

Copy the agent connection command displayed by Jenkins.

It will look like:

```
None
java -jar agent.jar \
-url http://localhost:8080/ \
-secret YOUR_SECRET_KEY \
-name slave1 \
-webSocket \
-workDir "/home/yourusername/jenkins-agent"
```

Replace `YOUR_SECRET_KEY` with the actual secret shown by Jenkins.

Run this command in the Ubuntu terminal.

Expected: Agent successfully connects to the Master.

Step 7: Verify Agent

Open:

Manage Jenkins → Nodes → slave1

Expected status: Online.

Agent connected successfully
Part 3: Creating and Executing a Jenkins Job on the Slave

In Part 2, we connected the Slave Agent to the Master.

Now we need to prove that the Slave can execute build jobs.

Step 8: Create a Freestyle Project

1. Open Jenkins Dashboard.
2. Click New Item.
3. Enter the project name:

```
None
Slave_function
```

4. Select Freestyle project.
5. Click OK.

Step 9: Configure the GitHub Repository

We will use a simple Python program stored in GitHub.

First, create a file named `app.py`:

```
None
print("Hello from Jenkins Slave!")print("Build executed successfully.")
```

Push this file to a GitHub repository.

Example repository URL:

```
None
https://github.com/USERNAME/jenkins-agent-demo.git
```

Now in Jenkins:

1. Open the `Slave_function` project.
2. Click Configure.
3. Find Source Code Management.
4. Select Git.
5. Enter your GitHub repository URL.

Click Save after configuring the project.

Why are we using GitHub?

To allow Jenkins to download the source code before executing the build.

Step 10: Add a Build Step

Open the project configuration again.

Under Build Steps:

1. Click Add build step.
2. Select Execute shell.
3. Enter these commands:

```
None
echo "Executing build on Jenkins Slave"

hostname

java -version

python3 app.py
```

Understand the commands:

Command	Purpose
`echo`	Displays a message
`hostname`	Displays the machine name
`java -version`	Checks Java
`python3 app.py`	Executes Python program

Step 11: Restrict the Job to Slave

This is the MOST IMPORTANT step.

In the project configuration:

1. Go to General.
2. Enable Restrict where this project can be run.
3. Enter:

```
None
slave1
```

4. Click Save.

Why do we enter slave1?

Because `slave1` is the label assigned to our Agent node.

This tells Jenkins to execute the job on that Agent instead of the built-in controller node.

Step 12: Execute the Build

1. Open the `Slave_function` project.
2. Click Build Now.
3. Go to Build History.
4. Click the latest build number.
5. Select Console Output.

Expected output:

```
None
Started by user admin

Building remotely on slave1

Executing build on Jenkins Slave

slave1

openjdk version "17..."

Hello from Jenkins Slave!
Build executed successfully.

Finished: SUCCESS
```

Expected result: SUCCESS

The exact output may differ depending on your username, hostname and Java version.

Part 4: Understand the complete experiment

Final execution flow

Jenkins Master

Manages and schedules jobs

Assigns job using label slave1

Jenkins Slave

Executes the build

Build Successful

Finished: SUCCESS

EXPERIMENT – 6

Title: Docker Architecture and Container Life Cycle

1. Aim: To understand Docker Architecture and Container Life Cycle, install Docker and execute Docker commands to manage images and interact with containers.

2. Objectives

- To understand the basic concepts and architecture of Docker.
- To study the different stages of the Docker Container Life Cycle.
- To install and configure Docker on Ubuntu.
- To download, list and manage Docker images.
- To create, run, stop, restart and remove Docker containers.
- To execute Docker commands and interact with containers.

3. Theory

Docker is an open-source containerization platform used to develop, package and run applications in isolated environments called containers.

Docker allows applications and their dependencies to be packaged together, making them portable and consistent across different environments.

Docker Architecture

Docker follows a client-server architecture consisting of the following components:

1. Docker Client: It is the interface through which users communicate with Docker using commands such as `docker run`, `docker pull` and `docker ps`.

2. Docker Daemon: It is a background service responsible for managing Docker images, containers, networks and volumes.

3. Docker Images: Images are read-only templates used to create Docker containers.

4. Docker Containers: Containers are lightweight, isolated environments created from Docker images to run applications.

5. Docker Registry: It is a storage location used to store and distribute Docker images. Docker Hub is a popular public registry.

Docker Container Life Cycle

The Docker Container Life Cycle describes the different states of a container from creation to deletion.

The main stages are:

1. Created: A container is created but not yet started.
2. Running: The container is actively executing.
3. Paused: The processes inside the container are temporarily suspended.
4. Stopped: The container execution is stopped.
5. Deleted: The container is permanently removed.

Docker provides commands such as `docker run`, `docker stop`, `docker start` and `docker rm` to manage the container life cycle.

5. Procedure

Step 1: Check Docker installation

Open the Ubuntu terminal and execute:

Command:

```
None
docker --version
```

Expected Output:

```
None
Docker version 29.1.3
```

This confirms that Docker is installed on Ubuntu.

Step 2: Check Docker service status

Command:

```
None
sudo systemctl status docker
```

Expected Output:

```
None
docker.service - Docker Application Container Engine
Loaded: loaded
Active: active (running)
```

This confirms that the Docker service is running.

Press q to exit the status screen.

Step 3: Download the Nginx image

Command:

```
None
sudo docker pull nginx
```

Expected Output:

```
None
Using default tag: latest
latest: Pulling from library/nginx
Pull complete
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

This downloads the Nginx image from Docker Hub.

Step 4: List Docker images

Command:

```
None
sudo docker images
```

Expected Output:

```
None
IMAGE   ID       SIZE
nginx   <image-id>   <image-size>
```

This displays the downloaded Docker images.

Step 5: Create and run a container

Command:

```
None
sudo docker run -d --name mynginx -p 8090:80 nginx
```

Expected Output:

```
None
<container-id>
```

Explanation:

- `-d` : Runs the container in the background.
- `--name mynginx` : Assigns the name mynginx.
- `-p 8090:80` : Maps host port 8090 to container port 80.
- `nginx` : Specifies the Docker image.

This creates and starts an Nginx container.

Step 6: Verify the running container

Command:

```
None
sudo docker ps
```

Expected Output:

```
None
CONTAINER ID   IMAGE   COMMAND   STATUS       PORTS
<container-id> nginx   ...       Up           0.0.0.0:8090->80/tcp
```

This confirms that the container is running.

Step 7: Access the web server

Open Firefox and enter the following URL:

```
None
http://localhost:8090
```

Expected Output:

The Nginx welcome page appears in the browser.

This confirms that the Nginx container is successfully serving web content.

Step 8: View all containers

Command:

```
None
sudo docker ps -a
```

Expected Output:

```
None
CONTAINER ID   IMAGE   STATUS       NAMES
<container-id> nginx   Up           mynginx
```

This displays all containers, including running and stopped containers.

Step 9: Inspect the container

Command:

```
None
sudo docker inspect mynginx
```

Expected Output:

Displays detailed information about the container, including its ID, configuration, network settings, image and status.

This helps to understand the internal configuration of a Docker container.

Step 10: Stop the container

Command:

```
None
sudo docker stop mynginx
```

Expected Output:

```
None
mynginx
```

This stops the running container.

Step 11: Restart the container

Command:

```
None
sudo docker start mynginx
```

Expected Output:

```
None
mynginx
```

Verify the status:

```
None
sudo docker ps
```

The container should appear in the running containers list.

This restarts the stopped container.

Step 12: View container logs

Command:

```
None
sudo docker logs mynginx
```

Expected Output:

Displays the logs generated by the Nginx container.

This helps to monitor container activities and troubleshoot problems.

Step 13: Stop and remove the container

First, stop the container:

```
None
sudo docker stop mynginx
```

Then remove the container:

```
None
sudo docker rm mynginx
```

Expected Output:

```
None
mynginx
```

This permanently removes the container.

Step 14: Remove the Docker image

Command:

```
None
sudo docker rmi nginx
```

Expected Output:

```
None
Untagged: nginx:latest
Deleted: sha256:<image-id>
```

This removes the downloaded Nginx image from the local system.

Step 15: Verify image removal

Command:

```
None
sudo docker images
```

Expected Output:

The Nginx image will no longer appear in the image list.

This confirms that the Docker image has been successfully removed.

6. Result

Thus, Docker was studied and installed successfully on Ubuntu. Docker architecture and container life cycle were understood, and various Docker commands were executed to download and manage images, create and run containers, access a web server, inspect containers, view logs, stop, restart and remove containers.

Step 1: Update Ubuntu packages

Command:

```
None
sudo apt update
```

This updates the package information of Ubuntu.

Step 2: Install Docker

Command:

```
None
sudo apt install docker.io -y
```

This installs Docker Engine on Ubuntu.

Step 3: Start Docker service

Command:

```
None
sudo systemctl start docker
```

This starts the Docker service.

Step 4: Enable Docker service

Command:

```
None
sudo systemctl enable docker
```

This enables Docker to start automatically when the system boots.

Step 5: Verify Docker installation

Command:

```
None
docker --version
```

Expected Output:

```
None
Docker version 29.1.3
```

This confirms that Docker is successfully installed.

Step 6: Check Docker service status

Command:

```
None
sudo systemctl status docker
```

Expected Output:

```
None
Active: active (running)
```

This confirms that the Docker service is running.

Press `q` to exit.

Experiment 7 — Complete execution flow

Step 1: Check Docker installation

```
None
docker --version
```

Checks whether Docker is installed.

Step 2: Create the project directory

```
None
mkdir docker-webapp
cd docker-webapp
```

- `mkdir` creates a directory.
- `cd` enters that directory.

Step 3: Create the HTML file

```
None
nano index.html
```

Enter this HTML content and save the file:

```
None
<!DOCTYPE html>
<html>
<head>
    <title>Docker Web Application</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
    <p>This web application is running inside a Docker container.</p>
</body>
</html>
```

This is the webpage that Nginx will serve.

Step 4: Create the Dockerfile

```
None
nano Dockerfile
```

Enter these instructions:

```
None
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Understand each line:

Instruction	What it does
`FROM nginx:latest`	Uses the latest Nginx image as the base.
`COPY index.html /usr/share/nginx/html/index.html`	Copies our webpage into Nginx's default web directory.
`EXPOSE 80`	Documents that Nginx listens on port 80.

`CMD ["nginx", "-g", "daemon off;"]`	Starts Nginx in the foreground when the container starts.

Important: The filename must be exactly `Dockerfile`, without a `.txt` extension.

Step 5: Check the project files

```
None
ls
```

Expected files:

```
None
Dockerfile
index.html
```

Step 6: Build the Docker image

```
None
docker build -t docker-webapp:latest .
```

- `docker build` builds an image using the Dockerfile.
- `-t` assigns a name and tag.
- `docker-webapp` is the image name.
- `latest` is the tag.
- `.` specifies the current directory as the build context.

Step 7: Verify the image

```
None
docker images
```

This lists locally available Docker images. Look for `docker-webapp` with the `latest` tag.

Step 8: Run the container

```
None
docker run -d --name docker-webapp-container -p 8090:80 docker-webapp:latest
```

Understand the command:

- `-d` runs the container in the background.
- `--name` assigns a name to the container.
- `-p 8090:80` maps host port `8090` to container port `80`.
- `docker-webapp:latest` specifies the image to run.

Step 9: Check the running container

```
None
docker ps
```

Displays running containers.

Step 10: Access the web application

Open your browser and enter:

```
None
http://localhost:8090
```

You should see Hello from Docker!

Step 11: Check container logs

```
None
docker logs docker-webapp-container
```

Displays the container's logs, which help you inspect its activity and diagnose problems.

Step 12: Stop the container

```
None
docker stop docker-webapp-container
```

Stops the running container.

Step 13: Start the container again

```
None
docker start docker-webapp-container
```

Starts the existing stopped container.

Verify it:

```
None
docker ps
```

Step 14: Remove the container

```
None
docker stop docker-webapp-container
docker rm docker-webapp-container
```

Verify removal:

```
None
docker ps -a
```

The removed container should no longer appear. The image remains available unless you remove it separately.

exp-8

1. What are Docker and Kubernetes?

Imagine you developed a small website that displays:

- Hello from Docker!
- This web application is running inside a Docker container.

You want to run this website reliably. Docker and Kubernetes help with different parts of that task.

Docker — runs the application in a container

Docker packages your application and its required files into an image. It can use that image to create and run containers.

Kubernetes — manages containers

Kubernetes helps run applications across a cluster, maintain the required number of copies, expose them through networking, and restart or replace failed Pods when possible.

Simple example: Docker helps run your website in a container. Kubernetes helps manage two or more copies of that website.

2. Important Kubernetes terminology

Learn these concepts in this order.

1. Docker image

An image is a packaged template containing the application files and instructions needed to run it.

Example: `docker-webapp:latest`

2. Kubernetes cluster

A cluster is a group of computing resources managed by Kubernetes. In our experiment, Minikube creates a local cluster on one computer.

3. Pod

A Pod is the smallest deployable unit in Kubernetes. It runs one or more containers. In our experiment, each Pod runs one Nginx container serving our website.

4. Deployment

A Deployment manages Pods and maintains the desired number of replicas. If we request two replicas, Kubernetes attempts to keep two Pods running.

5. Service

A Service provides a stable network endpoint for accessing the application Pods. Our experiment uses a `NodePort` Service.

3. How these components work together

Docker image

Contains the Nginx web application

Minikube cluster

Runs Kubernetes on your computer

Deployment

Requests two replicas

Pod 1

Nginx container

Pod 2

Nginx container

NodePort Service

Provides access to the application through a browser

For example, if one Pod fails, the Deployment can create a replacement to return to the desired number of replicas. The Service directs traffic to eligible Pods.

4. What are `kubectl` and Minikube?

- Minikube: Creates and runs a local Kubernetes cluster for learning and development.
- `kubectl`: The command-line tool used to communicate with Kubernetes and manage its resources.
- Docker: Builds images and runs containers. In this experiment, it also provides the driver for Minikube.

A cluster typically has a control plane, which manages the cluster, and worker nodes, which run application workloads. Minikube can run these roles together on one local node.

5. Theory you can write in your record

Kubernetes is an open-source container orchestration platform used to deploy, manage, and scale containerized applications. It automates container scheduling, scaling, service discovery, load balancing, and recovery of failed application Pods.

A Kubernetes cluster consists of a control plane and worker nodes. The control plane manages the cluster, while worker nodes run application workloads inside Pods.

The main components used in this experiment are:

- Pod: The smallest deployable unit in Kubernetes, containing one or more containers.
- Deployment: Maintains the desired number of Pod replicas and replaces failed Pods.
- Service: Provides a stable network endpoint for accessing application Pods.
- Minikube: Runs a local Kubernetes cluster for learning and development.
- kubectl: A command-line tool used to interact with the Kubernetes cluster.
- Docker image: A packaged template used to create containers.

The experiment deploys a sample Nginx web application using a Kubernetes Deployment with two replicas. A NodePort Service exposes the application so that it can be accessed using a web browser. Finally, the number of replicas is increased and decreased to demonstrate scaling.

6. What if Docker, kubectl, or Minikube is not installed?

Your screenshots show Ubuntu 22.04, so the following commands are for an Ubuntu lab terminal. Run the verification commands first. Do not reinstall tools that are already installed.

Step 1: Check all three tools

Run these commands one by one:

```
None
docker --version
kubectl version --client
minikube version
```

If a command displays a version, that tool is installed. If you see `command not found`, it is missing or not on the system's PATH.

Step 2: Install Docker if missing

Docker is the container engine needed for our Minikube Docker driver. The following installs Ubuntu's packaged Docker Engine:

```
None
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable --now docker
docker --version
```

If `docker --version` works but Docker commands report permission denied, check:

```
None
sudo systemctl status docker
```

If your account lacks Docker permissions, you may need your lab administrator to configure access. Avoid changing permissions on a shared lab system without approval.

Official reference: Docker installation for Ubuntu.

GitHub

Step 3: Install kubectl if missing

The easiest option on Ubuntu is Snap:

```
None
sudo snap install kubectl --classic
```

Verify:

```
None
kubectl version --client
```

Official reference: Install kubectl on Linux.

Kubernetes

Step 4: Install Minikube if missing

Run:

```
None
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Verify:

```
None
minikube version
```

These commands download the Linux AMD64 binary. They assume the lab computer uses an AMD64 processor and has `curl` installed.

Official reference: Minikube installation and startup.

minikube

Step 5: Start Minikube

Once Docker, `kubectl`, and Minikube are installed:

```
None
minikube start --driver=docker
```

Check:

```
None
minikube status
kubectl get nodes
```

You want to see a running cluster and a node with status `Ready`.

Minikube needs sufficient memory, CPU, disk space, and internet access for initial setup. If the lab computer restricts installations or administrator privileges, ask the lab technician to prepare the environment beforehand.

minikube+1

7. Complete experiment execution — from the beginning

Now assume the tools are installed and you are ready to execute the experiment.

Step 1: Verify the environment

```
None
docker --version
kubectl version --client
minikube version
```

Meaning: These commands check that the required software is available.

Step 2: Start the cluster

```
None
minikube start --driver=docker
```

Meaning: Starts a local Kubernetes cluster using Docker.

Then:

```
None
kubectl get nodes
```

Meaning: Lists the nodes in the cluster. Confirm that the node is `Ready`.

Step 3: Prepare the web application

If your previous Docker experiment's files are still present, reuse them. Otherwise, create a sample web application now.

Create a folder:

```
None
mkdir -p ~/docker-webapp
cd ~/docker-webapp
```

Create the HTML file:

```
None
nano index.html
```

Type:

```
None
<!DOCTYPE html>
<html>
<head>
    <title>Docker Web Application</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
    <p>This web application is running inside a Docker container.</p>
```

```
    <p>Kubernetes deployment successfully completed.</p>
</body>
</html>
```

Save using `Ctrl + O`, press `Enter`, then exit using `Ctrl + X`.

Create the Dockerfile:

```
None
nano Dockerfile
```

Type:

```
None
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Save and exit.

Meaning of the Dockerfile:

- `FROM nginx:latest` uses the Nginx web server image as the base.
- `COPY` copies your HTML file into the directory served by Nginx.
- `EXPOSE 80` documents the port used by the web server.

Step 4: Build the Docker image

Make sure you are inside `~/docker-webapp`, then run:

```
None
docker build -t docker-webapp:latest .
```

Meaning: Builds a Docker image named `docker-webapp` with the tag `latest`. The dot (`.`) means the current directory is the build context.

Check the image:

```
None
docker images
```

You should see `docker-webapp` in the image list.

Step 5: Load the image into Minikube

```
None
minikube image load docker-webapp:latest
```

Meaning: Makes your locally built image available to the Minikube node. This avoids needing to push the image to Docker Hub.

Step 6: Create the Deployment file

Create a separate folder for Kubernetes configuration:

```
None
mkdir -p ~/kubernetes-lab
cd ~/kubernetes-lab
nano deployment.yaml
```

Enter the following configuration:

```
None

apiVersion: apps/v1
kind: Deployment
metadata:
  name: docker-webapp-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: docker-webapp
  template:
    metadata:
      labels:
        app: docker-webapp
    spec:
      containers:
        - name: docker-webapp
          image: docker-webapp:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 80
```

Save and exit Nano.

Meaning: This file tells Kubernetes to run two copies of the web application. The selector and labels connect the Deployment to its Pods.

Step 7: Deploy the application

```
None
kubectl apply -f deployment.yaml
```

Meaning: Reads the YAML file and creates or updates the Deployment in Kubernetes.

Step 8: Verify the Deployment

```
None
kubectl get deployments
```

Look for `docker-webapp-deployment` with `2/2` ready replicas.

If it shows `0/2`, don't proceed immediately. Check the Pods and troubleshoot the problem first.

Step 9: Verify the Pods

```
None
kubectl get pods
```

Meaning: Lists the Pods created by the Deployment.

Expected: two Pods with `Running` status and `1/1` readiness.

Step 10: Create the Service file

```
None
nano service.yaml
```

Enter:

```
None

apiVersion: v1
kind: Service
metadata:
  name: docker-webapp-service
spec:
  type: NodePort
  selector:
    app: docker-webapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Save and exit.

Meaning: The Service uses the `app: docker-webapp` label to find the Pods and exposes their port 80 through a NodePort.

Step 11: Apply the Service

```
None
kubectl apply -f service.yaml
```

Meaning: Creates the Service using the configuration file.

Step 12: Verify the Service

```
None
kubectl get services
```

Look for `docker-webapp-service` with type `NodePort`.

Step 13: Open the application

```
None
minikube service docker-webapp-service
```

Meaning: Finds the Service's access URL and normally opens it in a browser. If the browser does not open automatically, use the URL printed by the command.

You should see your HTML page.

Step 14: Verify the HTML inside a Pod

First, list the Pods:

```
None
kubectl get pods
```

Copy one complete Pod name and run:

```
None
kubectl exec -it <pod-name> -- cat /usr/share/nginx/html/index.html
```

Replace `<pod-name>` with the actual name; do not type the angle brackets.

Meaning: Runs the `cat` command inside the Pod to display the HTML file stored in the container.

Step 15: Test scaling

Increase the replicas from two to three:

```
None
kubectl scale deployment docker-webapp-deployment --replicas=3
```

Verify:

```
None
kubectl get pods
```

You should see three Pods after they are created and become ready.

Restore the original number:

```
None
kubectl scale deployment docker-webapp-deployment --replicas=2
```

Verify again:

```
None
kubectl get pods
```

Meaning: Scaling changes the desired number of Pod replicas. The Deployment controller works to maintain that number.

8. Troubleshooting: what if something goes wrong?

These are the common errors you should know for the lab exam.

Problem	What to do
`command not found`	The tool may be missing or not in PATH. Check installation and permissions.
Docker permission denied	Check `sudo systemctl status docker`; ask the lab administrator to configure access if needed.
Minikube fails to start	Check Docker status, available memory, disk space, and internet connectivity.
`ImagePullBackOff` or image not found	Check the image name, then run `minikube image load docker-webapp:latest` again.
YAML parsing error	Check spaces and indentation in `deployment.yaml` or `service.yaml`.
Pods are not ready	Run `kubectl describe pods` and inspect the Events section for errors.
Browser does not open	Run `minikube service docker-webapp-service` again and use the URL it provides.

If a Pod is not starting, these commands are especially useful:

```
None
kubectl get pods
kubectl describe pods
kubectl get events
```

Aim: To install and configure Pull-based Software Configuration Management and provisioning tools using Puppet.

Procedure

Step 1: Update package list

```
None
sudo apt update
```

Step 2: Download Puppet repository package

For Ubuntu 22.04 (Jammy):

```
None
wget https://aptpuppetcore.puppet.com/public/puppet8-release-jammy.deb
```

Step 3: Install Puppet repository

```
None
sudo dpkg -i puppet8-release-jammy.deb
sudo apt update
```

Step 4: Install Puppet Agent

```
None
sudo apt install puppet-agent -y
```

Step 5: Check Puppet version

```
None
/opt/puppetlabs/bin/puppet --version
```

Step 6: Install Puppet Server

```
None
sudo apt install puppetserver -y
```

Step 7: Start and check Puppet Server

```
None
sudo systemctl start puppetserver
sudo systemctl status puppetserver
```

Press `q` to exit the status screen.

Step 8: Configure Puppet hostname

```
None
echo "127.0.0.1 puppet" | sudo tee -a /etc/hosts
```

Step 9: Verify hostname

```
None
getent hosts puppet
```

Step 10: Check Puppet Server port

```
None
sudo ss -ltnp | grep 8140
```

Step 11: Create a Puppet manifest

```
None
nano first.pp
```

Enter:

```
None
file { '/tmp/puppet-demo':
  ensure => directory,
}
```

Save using `Ctrl + O`, Enter, and exit using `Ctrl + X`.

Step 12: Apply the manifest

```
None
sudo /opt/puppetlabs/bin/puppet apply first.pp
```

Step 13: Verify the directory

```
None
ls -ld /tmp/puppet-demo
```

Step 14: Check idempotency

```
None
sudo /opt/puppetlabs/bin/puppet apply first.pp
```

The second run should report that no changes are needed.

Step 15: Run Puppet Agent

```
None
sudo /opt/puppetlabs/bin/puppet agent --test
```

Output

- Puppet Agent and Puppet Server are installed.
- Puppet Server is started and listening on port 8140.
- The Puppet manifest creates `/tmp/puppet-demo`.
- Reapplying the manifest confirms idempotency.

Result

Puppet was installed and a sample manifest was executed successfully to demonstrate configuration management.

exp-10

1. Aim

To study and implement software configuration management and provisioning using Puppet manifests, modules, classes, and functions in Ubuntu Linux.

2. Theory

What is Puppet?

Puppet is a configuration management and provisioning tool used to automate the configuration of computers and servers.

Suppose you have 100 computers in a laboratory. You need to install software and create the same configuration on every computer. Doing this manually takes a lot of time.

Puppet helps automate this process by allowing you to write instructions in code.

For example, you can tell Puppet to:

- Install a software package such as `curl`.
- Create a file in a specific location.
- Write content into that file.
- Configure services and maintain the desired system state.

Important Puppet concepts

1. Manifest (`.pp`)

A manifest is a file containing Puppet instructions. It specifies the desired state of resources.

Example: `site.pp` tells Puppet to install `curl` and create a text file.

2. Module

A module organizes Puppet code into a standard directory structure so that configuration can be reused and maintained easily.

Example: a `puppet_demo` module stores its main class in `manifests/init.pp`.

3. Class

A class groups related resources into a reusable unit.

Example: `demo_class` creates a file and writes a message into it.

4. Function

A function performs an operation in Puppet code.

Example: `notice('Hello')` displays a message during Puppet catalog compilation.

How Puppet works

Puppet Manifest

Instructions written in a `.pp` file

Puppet Apply

Reads and processes the instructions

System Configuration

Installs packages or creates files

Verify the Result

Use `cat` or other commands to check

For this experiment, we use `puppet apply` to execute the manifests locally. This means you do not need a separate Puppet agent-server connection for these demonstrations, although you will check the Puppet server status as part of the initial setup.

3. Procedure — Step-by-step execution

Step 1: Verify Puppet installation

Open the Ubuntu terminal using `Ctrl + Alt + T`.

First, check whether Puppet is installed:

```
None
sudo /opt/puppetlabs/bin/puppet --version
```

Expected output will be a version number, such as:

```
None
8.22.0
```

Next, check the Puppet server:

```
None
sudo systemctl status puppetserver --no-pager
```

Look for `active (running)` in the output.

Remember: If Puppet is already installed from the previous experiment, you do not need to install it again. If the server is not running, you can still perform these local `puppet apply` demonstrations as long as the Puppet executable is available.

Step 2: Create the experiment directory

Type these commands one by one:

```
None
mkdir -p ~/puppet-experiment
cd ~/puppet-experiment
```

Meaning:

- `mkdir -p` creates the directory, including any missing parent directories.
- `~` means your home directory.
- `cd` moves into the directory.

Check your current location:

```
None
pwd
```

Expected output resembles:

```
None
/home/your-username/puppet-experiment
```

Step 3: Create and execute a Puppet manifest

Concept: A manifest contains instructions that tell Puppet what resources should exist and how they should be configured.

Create the file:

```
None
nano site.pp
```

Type the following code exactly:

```
None

package { 'curl':
  ensure => installed,
}

file { '/tmp/puppet-demo.txt':
  ensure  => file,
  content => "Puppet manifest executed successfully.\n",
}
```

Save the file in Nano:

1. Press `Ctrl + O`.
2. Press `Enter`.
3. Press `Ctrl + X`.

Now execute the manifest:

```
None
sudo /opt/puppetlabs/bin/puppet apply site.pp
```

Verify the output file:

```
None
cat /tmp/puppet-demo.txt
```

Expected output:

```
None
Puppet manifest executed successfully.
```

What did we do?

- `package` tells Puppet to ensure that `curl` is installed.
- `file` tells Puppet to manage a file.
- `ensure => file` means the file must exist.
- `content` specifies the text written into the file.
- `puppet apply` processes and applies the configuration.

The `\n` adds a newline at the end of the message.

Step 4: Create and execute a Puppet class

Concept: A class groups related Puppet resources into a reusable block.

Create the file:

```
None
nano class_demo.pp
```

Type:

```
None

class demo_class {
  file { '/tmp/puppet-class.txt':
    ensure  => file,
    content => "Puppet class executed successfully.\n",
  }
}

include demo_class
```

Save using `Ctrl + O`, `Enter`, and `Ctrl + X`.

Execute:

```
None
sudo /opt/puppetlabs/bin/puppet apply class_demo.pp
```

Verify:

```
None
cat /tmp/puppet-class.txt
```

Expected output:

```
None
Puppet class executed successfully.
```

Understand the code:

- `class demo_class { ... }` defines a class.
- The `file` resource describes what the class should configure.
- `include demo_class` declares the class so Puppet applies its configuration.

Memory trick: Define the class first, then include it.

Step 5: Create a Puppet module

Concept: A module organizes Puppet configuration into a standard directory structure. A module can contain classes, manifests, templates, and files.

Create the module directories:

```
None
mkdir -p ~/puppet-experiment/modules/puppet_demo/manifests
```

Create the module's main manifest:

```
None
nano ~/puppet-experiment/modules/puppet_demo/manifests/init.pp
```

Type:

```
None

class puppet_demo {
  file { '/tmp/puppet-module.txt':
    ensure  => file,
    content => "Puppet module executed successfully.\n",
  }
}
```

Save the file.

Why is the file named `init.pp`?

In Puppet's standard module structure, `manifests/init.pp` contains the module's main class. For the module named `puppet_demo`, the class is named `puppet_demo`.

Step 6: Apply the Puppet module

Create a manifest that includes the module's class:

```
None
nano ~/puppet-experiment/module_demo.pp
```

Type:

```
None

include puppet_demo
```

Save the file.

Now execute these commands:

```
None
cd ~/puppet-experiment
```

```
None
sudo /opt/puppetlabs/bin/puppet apply \
  --modulepath="$HOME/puppet-experiment/modules" \
  module_demo.pp
```

The command is split across three lines for readability. Type the backslash `\` at the end of the first two lines exactly as shown, or type the entire command on one line:

```
None
sudo /opt/puppetlabs/bin/puppet apply --modulepath="$HOME/puppet-experiment/modules"
module_demo.pp
```

Verify the result:

```
None
cat /tmp/puppet-module.txt
```

Expected output:

```
None
Puppet module executed successfully.
```

Understand the important option:

`--modulepath` tells Puppet where to search for modules. Here, Puppet finds the `puppet_demo` module, loads its class, and creates the file.

The execution flow is:

`modules/puppet_demo/manifests/init.pp`

`module_demo.pp` includes `puppet_demo`

`/tmp/puppet-module.txt` is created

Step 7: Demonstrate a Puppet function

Concept: Puppet functions perform operations during catalog compilation or application. The built-in `notice()` function displays a message in the terminal.

Create the file:

```
None
nano ~/puppet-experiment/function_demo.pp
```

Type:

```
None

notice('Puppet function executed successfully.')

file { '/tmp/puppet-function.txt':
  ensure  => file,
  content => "Puppet function demonstration completed.\n",
}
```

Save the file and execute:

```
None
sudo /opt/puppetlabs/bin/puppet apply function_demo.pp
```

Verify:

```
None
cat /tmp/puppet-function.txt
```

Expected output:

```
None
Puppet function demonstration completed.
```

Understand the code:

- `notice()` displays a message during Puppet processing.
- `file` creates the output file.
- `ensure => file` ensures that the file exists.
- `content` specifies what gets written inside it.

Notice that the `notice()` function and the `file` resource are two separate instructions. The function prints a message; the file resource creates the file.

4. Result

Puppet was used to demonstrate software configuration management and provisioning in Ubuntu Linux. A Puppet manifest was created and executed to install a package and create a file. Classes, modules, and functions were demonstrated successfully by creating and verifying their respective output files.